I need to fix Task 1: Refine Collision Detection in my Pygame Marble Tilt Maze project.

In game/game_engine.py, the _resolve_wall_collisions() method currently uses marble_rect.colliderect(wall_rect). This treats the circular marble as a square and causes false collisions near wall corners.

Modify only the collision detection logic for this task.

Requirements:

1. Implement accurate circle-versus-rectangle collision detection by finding the closest point on each wall rectangle to the marble's center and checking whether the distance is less than the marble's radius.
2. Resolve actual collisions by moving the marble out of the wall and reflecting only the velocity component along the collision normal, using a reasonable bounce factor.
3. Avoid false collisions when the marble's bounding square overlaps a wall but the actual circle does not touch it.
4. Handle wall corners correctly, including cases where the marble center is diagonally outside a rectangle.
5. Preserve the existing maze, controls, timer, goal behavior, and other game features.
6. Do not implement the game-over screen, replay options, difficulty selection, or sound effects yet.
7. Explain the changes and provide manual test cases for straight wall impacts, corner approaches, and outer boundaries.
   Inspect the existing code before editing. Make the smallest reliable change and do not rewrite unrelated files.

I'll inspect the collision method and nearby game/test structure first, then make a focused change in `game_engine.py` and run the smallest relevant validation.

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze\\game\\game_engine.py"

}

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze"

}

Find files matching `**/*test*`

Search for `_resolve_wall_collisions|colliderect|class Marble|class Wall`

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze\\game\\marble.py"

}

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze\\game\\wall.py"

}

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze\\README.md"

}

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze\\requirements.txt"

}

Read

Completed with input: {

"path": "C:\\Users\\rishi\\42_marble-tilt-maze\\main.py"

}

The current resolver only uses axis-aligned overlap depths. I'll replace that method's collision test with closest-point circle/rectangle geometry, including a defined inward recovery direction when the center has entered an outer-boundary rectangle; the rest of the engine stays untouched.

Edit

Ran terminal command: git diff --check; git diff -- game/game_engine.py

Checked , 1 problem found

Updated only `_resolve_wall_collisions()` in `game_engine.py`.

The resolver now checks the distance from the marble's center to the closest point on each wall, so bounding-square overlaps alone don't trigger collisions. It pushes overlapping marbles out along the collision normal and reflects only velocity into that normal, with a 0.3 bounce factor. When the center is inside a rectangle, it selects an exit direction; outer boundary walls push back toward the playable area.

Manual test cases

- **Straight wall impact:** Steer directly into an internal wall. Confirm the marble is pushed clear and bounces back, while motion parallel to the wall is preserved.

- **Corner approach:** Approach an internal wall's end diagonally. Confirm there's no bounce while only the marble's bounding square overlaps, then a diagonal bounce once the circle reaches the corner.

- **Outer boundaries:** Steer into each board edge and a board corner. Confirm the marble stays within the playable area and bounces inward.

`git diff --check` passed. No automated tests were found; the selected Python environment reports that it cannot resolve `pygame`, so I couldn't run the game here.

Review the current _resolve_wall_collisions() method for correctness. The outer boundary walls are thin rectangles: top and bottom span the full screen width, and left and right span the full screen height. Correctly detect these wall orientations using their actual rectangle coordinates and dimensions, not assumptions that the rectangle itself spans both dimensions. Ensure that when the marble center is inside a wall rectangle, the resolution pushes it back into the playable area for outer boundaries and out through the nearest face for internal walls. Keep the circle-versus-rectangle closest-point test and do not change any other tasks or files. Explain the correction.

Corrected the center-inside resolution branch in game_engine.py. It now identifies horizontal versus vertical rectangles by their dimensions, then identifies an outer wall by its matching screen edge. This pushes the marble back into the playable area for outer boundaries; for internal walls, it still selects the nearest face and pushes the marble out through it.

The closest-point circle-versus-rectangle test and bounce behavior are unchanged. git diff --check passed. Pylance still reports that pygame cannot be resolved in the selected environment.

Implement Task 2 (Game Over Condition) from the README for this marble tilt maze game.

Requirements:

1. When the marble reaches the goal, display a clear game-over screen showing "Maze Solved!" and the finish time.
2. When the time limit expires, display a clear game-over screen showing "Time's Up!".
3. Render these messages inside the Pygame window, not just in the terminal.
4. Keep the game-over screen visible until the player closes the game window or presses an appropriate key.
5. Stop updating the marble and timer after the game ends.
6. Preserve the existing gameplay, collision detection, timer logic, and other features.
7. Make only the changes necessary for Task 2. Do not implement the replay option or sound effects yet.
8. Follow the existing code style and use the existing Pygame screen and font setup where appropriate.
   Inspect the existing code before editing. Explain which files you changed and why. Do not commit the changes.

Implemented Task 2 without committing.

- game_engine.py: Displays “Maze Solved!” with the finish time or “Time’s Up!” in the Pygame window. The result screen omits the timer and returns before drawing gameplay, while the existing update/input guards keep the marble and game state frozen.
- main.py: Lets Enter, Space, or Escape exit from the result screen and stops the loop cleanly.

Both files passed syntax validation, and git diff --check passed. I couldn’t run the game because Pygame is unresolved in the selected environment. No replay or sound features were added.

Implement Task 3 (Replay Option) from the project's README for this Pygame marble tilt maze.

First inspect the existing code in main.py, game/game_engine.py, game/marble.py, and game/wall.py.

Requirements:

1. After the player wins or times out, show options to Play Again or Exit.
2. Allow the player to choose Easy, Medium, or Hard difficulty before restarting.
3. Each difficulty must configure tilt strength, friction, and time limit. Use sensible, clearly defined values and keep them consistent throughout the game.
4. Restarting must reset the marble's position and velocity, timer, game-over state, result, finish time, and any other state needed for a fresh game.
5. The game must continue to support mouse-based tilting and preserve the existing collision detection and game-over screen.
6. Keep the controls clear and display the options inside the Pygame window.
7. Do not implement sound effects yet; that is Task 4.
8. Make only the changes necessary for Task 3. Preserve the existing code style and avoid unnecessary file changes.
9. Inspect the current game loop carefully so restarting does not require closing and reopening the application.
10. Do not commit the changes.
    After editing, explain which files changed, how the replay and difficulty selection work, and how to test each option.
11. Implemented Task 3 in `game_engine.py` only. The existing game loop already keeps processing events and rendering after a round ends, so it did not need changes.
12. After a win or timeout, press **Enter** or **Space** to open difficulty selection, then press **1** for Easy, **2** for Medium, or **3** for Hard. Press **Escape** on the result screen to exit, or on difficulty selection to return to the result screen.

Difficulty	Tilt strength	Friction	Time limit
Easy	0.4	0.03	60 seconds
Medium	0.6	0.02	45 seconds
Hard	0.85	0.01	30 seconds

13. Starting a round resets the marble position and velocity, timer, result, finish time, game-over state, and menu state. Mouse-based tilting and collision handling remain unchanged.

14. **Validation:** Syntax validation and a focused lifecycle check passed, covering selection of all three difficulties and fresh-round resets. Pygame could not be resolved in the selected environment, so I couldn’t verify the rendered menus in a live window.
15. **Manual checks:** Finish or time out a round; verify both menu screens and their controls. Choose each difficulty in separate rounds, then confirm the timer duration and that the marble starts at its original position with no residual velocity. Press Escape on each menu to confirm exit and back behavior. No commit was made.

Implement Task 4 (Sound Feedback) from the project's README.

First inspect the existing code, especially game/game_engine.py, main.py, and requirements.txt.

Requirements:

1. Play a sound effect when the marble collides with a wall.
2. Play a distinct sound effect when the player reaches the goal.
3. Play a distinct sound effect when the timer expires.
4. Use Pygame's audio facilities and initialize audio safely. The game should still work if audio initialization or sound loading fails.
5. Avoid playing the wall-bounce sound continuously every frame while the marble remains in contact with a wall. Play it only when an actual collision occurs.
6. Ensure the goal and timeout sounds each play only once per round.
7. Preserve all existing features: collision detection, the game-over screen, replay, and all three difficulty settings.
8. Do not implement unrelated features or commit the changes.
9. If sound assets are needed, use small, locally stored sound files and explain exactly where they should be placed. Do not rely on internet access at runtime. Inspect the code before editing. Explain which files changed and how to test each sound effect.

Implemented Task 4 in game_engine.py only. The game uses short, synthesized sounds through Pygame’s mixer, so **no sound assets or new dependencies are needed**.

- A wall sound plays when collision resolution detects an approaching impact, not merely while the marble overlaps a wall.
- Distinct goal and timeout sounds play once each per round. Replay resets their one-shot state.
- Mixer initialization and sound creation/playback failures print a warning and leave gameplay running.

Validation: Syntax validation and focused checks passed for wall-impact triggering, one-shot goal/timeout sounds, replay reset, and mixer initialization failure. I couldn’t test playback on real audio hardware because Pygame is unresolved in the selected environment.

Manual sound checks: Steer into a wall and listen for a single bounce sound; reach the goal and confirm the goal sound plays once; let the timer expire and confirm the distinct

timeout sound plays once. Replay and repeat to verify the result sounds can play again in a new round.